AI Agents and Agentic Workflows: Your 2026 Roadmap

1. Source Info

Title: AI Agents and Agentic Workflows: Your 2026 Roadmap (Maven Lightning Lesson) **Speakers:** Sara Davison and Tyler Fisk, Co-Founders, AI Build Lab **Date:** Late 2025 (December, before the January 2026 cohort launch) **Link:** maven.com/sara-davison/ai-build-lab

---

2. Executive Summary

Sara Davison and Tyler Fisk, who have graduated 1,300+ practitioners through their AI Build Lab programs on Maven, argue that the dominant skill in agentic AI is shifting from execution to orchestration. As drag-and-drop workflow assembly gets automated by co-pilot agents, the durable advantage moves to system architecture: designing agent teams, building multi-layer knowledge environments, running rigorous evaluations, and deeply understanding the business problem before touching a tool. They call this the Forward Deployed Engineer mindset, and the market agrees: FDE job listings are up 800% in the past year. Their evaluation framework, centered on a "goldens database" of A-plus agent performances, is what they consider one of the few genuinely defensible assets in an industry where models and platforms commoditize fast.

---

3. Key Insights and Frameworks

3.1 From the Tool Era to the Orchestration Era

Sara opens by naming the moment: practitioners are leaving an era of hands-on node-stitching and entering one where the value is in design, thinking, and architecture. Tyler illustrates with a concrete story: he fed Cassidy's new co-pilot agent a plain-language description of a complex evaluation workflow he had already documented, and the co-pilot assembled it in one shot. But the critical nuance both are careful to name is that this only worked because Tyler had already done months of upstream work: testing the assistant manually, running observational evaluations, writing detailed system instructions, documenting inputs, outputs, and database connections.

Tyler describes his current working style: multiple agents and platforms running simultaneously, hooks set up on his system to ping him audibly when each completes a task, with the goal of long-running agentic workflows executing overnight via Claude Code as the orchestration layer. He acknowledges he has not fully cracked overnight runs yet, but Sara calls this "Tyler in the eighth dimension" and says it will become the new normal for knowledge workers.

3.2 The Supply-Demand Gap

Sara cites that 94% of leaders face AI skill shortages. The US is heading toward a 2:1 ratio of AI jobs to qualified people. Biggest demand spikes: prompting, context engineering, agentic workflow design, and human-AI collaboration. Job titles that did not exist two years ago (AI agent manager, orchestration engineer, orchestration and experience architect) are showing up in listings.

Tyler makes the self-employed case: a student named Brian went through multiple layoffs, took the course, found a niche problem, and went from zero to seven figures in about six months with himself, a part-time assistant, and agents. Sara names the archetype: "small giants," nimble teams with outsized impact.

3.3 Evolution, Not Revolution

Sara describes the teaching philosophy she repeats every session: provide agents increasing

levels of autonomy only as you earn trust and verify reliability at each level. Their four-week foundations course is structured as a deliberate progression: week one resets terminology (what LLMs actually are, assistant vs. agent, prompt drift, prompt overfitting); week two covers prompt engineering; week three is evaluations; week four is when multiple agents come together.

In the advanced course, students do not start building agents until week four of ten. Students who shortcut this process produce what Sara calls "AI slop": systems that seem to work because the technology is capable but produce unreliable outputs because the builder never defined what good looks like.

3.4 The Forward Deployed Engineer

Sara and Tyler describe their pre-AI consulting work and say they were already practicing this without knowing its name: sitting with the business, deeply understanding the process, building only after. The original FDE concept: instead of sending your best salesperson to a client's headquarters, you send your best engineer to sit with the team there.

Tyler describes what this means in practice. You document every input an employee receives, every decision they make, who they hand off to, what they produce, and what good looks like to them. What is that person thinking when they perform the task? What is their background and education? Only with that understanding can you translate the work into an agent team that can replicate or improve it.

The FDE role growing as technology gets easier is the market's evidence for Sara's claim: "Just because the technology is getting easier and frictionless, the fundamentals really still stick."

3.5 Progressive Evaluations: The 1% Skill

Tyler calls their evaluation philosophy "weaponized OCD of quality assurance." As building an 80% MVP becomes trivially easy, the 80-to-100% gap is the whole game. He credits Rachel Woods (former Meta data scientist, founder of The AI Exchange) for sending them down this path before evaluations were widely discussed.

The progressive evaluation framework:

- Three-point grading: green (pass), yellow (needs review), red (failed). No finer granularity to start.
- Begin with exactly three criteria per workflow step. Add only when necessary; never exceed eight.
- If approaching eight criteria and struggling to grade, the agent is struggling too. Split the task into smaller chunks rather than adding more criteria.
- Use evaluations as a continuous improvement loop, not just a quality gate.
- Build a "goldens database": a growing, versioned library of A-plus agent performances. Before executing any task, agents query this database to learn from prior excellence on similar tasks.

The LLM numeric midpoint problem: Tyler describes a research paper (shared by his friend Hunter) about AI-simulated buyer intent in retail. Researchers gave agents a 1-5 scale to rate purchase likelihood. Result: scores concentrated at 3, not matching real-world distributions, because LLMs "vibe" to numeric midpoints. The fix: instead of having the LLM output a number, have it write a rich text explanation of its reasoning, convert that to an embedding, and run semantic similarity against a known-good human evaluation to derive the score. Output quality from this approach is significantly higher.

Their evaluation agent, "Evie," only works because humans ran the initial evaluations first. Tyler is explicit: "A lot of folks try and jump to LLM as a judge too quickly." The human work at the beginning is not a bottleneck to eliminate; it is the source of taste and nuance that makes the automated system trustworthy.

Why this matters strategically: "This builds your own proprietary dataset for your system, for your client systems, that is like one of the few defensible things you're going to have now in this industry." Models commoditize. The curated record of excellent agent performance for your specific use case, accumulated through rigorous human evaluation, is genuinely scarce and non-replicable.

3.6 Designing the Environment for Agents

Sara uses the "Professor" agent in their Slack community as the case study. The Professor is connected to a knowledge base of all class transcripts, Slack history, and course materials

(a year of continuously updated data) running on basic vector store RAG. The problem: when someone asks "what time is the next foundations class?", the Professor pulls context from an early 2025 cohort rather than the most recent one. It gives the right answer for the wrong time period.

The distinction is between basic RAG (connections by semantic similarity) and what Sara calls the Cairns architecture with multi-layer knowledge:

- A vector store for semantic retrieval (the basic RAG layer)
- A graph database layer for relational and temporal reasoning (understanding that "most recent" should override "most relevant by text similarity")
- A shorter-term persistent memory layer for things agents always need to know

Tyler connects this to MCP (model context protocol) and how agents with coding ability can build or retrieve tools for themselves in real time rather than loading every possible tool into context upfront. This reduces context bloat and improves performance on long-horizon tasks.

3.7 Multi-Agent Orchestration: The Melissa Chen Demo

Sara frames the evolution: from bolting a single agent onto a process, to building agent teams with an orchestrator managing coordination. The core insight is about chaos management. Excellent individual agents can still produce chaotic outputs without coordination. An orchestrator does not execute tasks; it routes work, manages handoffs, enforces quality gates, and decides when to surface outputs to a human.

Tyler notes this pattern is already visible in how Anthropic built Claude Code: an orchestration layer spinning up sub-agents for subtasks. Multi-agent orchestration is becoming a foundational skill, not an advanced one.

The demo (built for the Mindvalley Summit, 13,000 attendees) uses a fictional course creator named Melissa Chen, with four specialist agents and one orchestrator:

Pulse: Data scientist agent. Scheduled scans of Slack and connected systems, monitors KPIs (homework submission rates, engagement frequency, behavioral baselines per student), identifies anomalies, alerts the team.

Nova: Learning psychology agent. Connected to student profiles, homework submissions, course transcripts. When Pulse flags an anomaly, Nova analyzes the student's full arc (past behavior, personality profile from intake, content engagement) and develops a psychological

interpretation.

Sage: Community-facing agent. Interacts with students in Slack, answers questions, logs interactions, drafts outreach in Melissa's communication style.

Aria: Orchestrator. Reads outputs of Pulse, Nova, and Sage. Determines whether the situation meets the threshold for notifying Melissa, develops the action plan, assigns tasks back to sub-agents, monitors execution, reports to Melissa.

Demo scenario: Alex Rivera, a high achiever (posts frequently, submits early, 100% on-time rate), logs a breakthrough in week three: "60% of my tasks don't need me." Week four: a frustrated post ("How do I delegate when I know I'll just redo everything?"), then 72 hours of silence and her first missed submission.

Pulse detects: 40x deviation from posting baseline, first-ever missed submission. Writes a structured report, pings Nova. Nova identifies the "breakthrough then breakdown" pattern (pulling from course transcripts), noting Alex's most energized post came before an implementation exercise that collided with her deepest identity conflict around delegation. Recommends cognitive reframing, pings Sage. Sage drafts outreach using Melissa's style (motivational interviewing, anchoring in past success), pings Aria. Aria reviews the full chain, decides it meets the notification threshold, checks Melissa's calendar, sends a morning message: "Alex Rivera needs 15 minutes for a critical mindset intervention. The team has already deployed initial support."

Melissa's only action: reply asking Aria to book the Tuesday meeting, email Alex, and prepare a pre-call Google Doc. Aria executes all three and reports back.

Built on Cassidy, Slack, and Zapier; entirely no-code. Architecture deliberately includes an audit trail: all inter-agent communication is in the Slack channel, every action logged.

3.8 Model Selection: Ferrari First, Then Optimize

Tyler describes a two-phase approach:

Phase one (Ferrari): Start every new workflow with the best proprietary models available (at the time: Claude Sonnet 3.5, Gemini 1.5 Pro, GPT-4o). Not just for quality: state-of-the-art models, when forced to reason out loud, generate detailed documentation of how they achieved the performance. That documentation seeds phase two.

Phase two (optimization): Break the workflow into sub-tasks and test smaller, cheaper models on each. Tyler's math: "You could run 27 GPT-4o mini models to one run of Sonnet

3.5." When you have enough training data and chunk into smaller subtasks, three GPT-4o mini agents can replace one Sonnet step, running faster and sometimes better. He calls this "controlled chaos": parallel smaller-model runs filtered to the best output.

Open-source timeline: Tyler has been watching DeepSeek's OCR model (which he says "essentially 10x'd the context window of agents" by compressing data more efficiently) and Kimi K2 (benchmarking highly on reasoning). His projection: by May 2026, open-source models will compete with today's best proprietary, runnable nearly for free on your own infrastructure. He is setting up a NAS at home to experiment. The vision: a hybrid stack where most tasks run on local open-source inference, proprietary models reserved for tasks that genuinely need them, with data sovereignty increasingly factoring into what gets sent to the cloud.

3.9 LLM-Agnostic Thinking

Sara describes this as both a curriculum philosophy and a professional necessity. Your agent system should not be coupled to any specific model; swapping in a different model for a specific role should be a configuration change, not a rebuild. This is resilience against the pace of model releases, which makes any "best model" claim obsolete within months. It also means you can apply the Ferrari-first strategy (see 3.8) without restructuring the system.

The agnosticism extends to tools and platforms. What they are building toward is practitioners who think at the level of agent design and system architecture, then execute in whatever platform fits the use case, client stack, or budget.

---

4. Tactical Playbook

1. Start with a PRD for your agents, not a tool. Before opening any platform, write a plain-language product requirements document: what the agent does, its inputs, outputs, databases, and success criteria. The PRD quality is now the ceiling on agent quality. (See 3.1)

2. Run the FDE process before building. For any workflow: sit with the person doing the work. Document every input, decision, handoff, output, and quality standard. Do not begin agent design until you could pass a quiz on the process. (See 3.4)

3. Use the Ferrari-first model strategy. Prototype with the best model. Force it to reason out loud. Document good performance. Progressively test smaller models on sub-tasks. Deploy expensive models only where genuinely needed. (See 3.8)

4. Implement progressive evaluations. Three criteria to start, never more than eight. Green/yellow/red scoring. Have the agent write a text explanation before scoring; derive numeric scores via semantic similarity against human evals. Build a goldens database from day one. (See 3.5)

5. Do initial human evaluations yourself. Do not automate from day one. The first several iterations of human evaluation create the dataset that makes LLM-as-judge trustworthy. (See 3.5)

6. Design multi-layer knowledge architecture. For production agents, basic RAG is not enough. Layer in: vector store for semantic retrieval, graph database for relational and temporal reasoning, persistent memory for always-on context. (See 3.6)

7. Build with an orchestrator from the start. Even with only two or three specialist agents, introduce a router early. Without it, capable agents produce chaotic outputs. (See 3.7)

8. Show up with a working mock-up. Sara and Tyler make this a core technique: a working prototype changes the conversation. The barrier to building one is now extremely low. (See 3.1)

9. Design human-in-the-loop checkpoints explicitly. Decide in advance which decisions require human approval, which outputs get reviewed before external sending, which anomaly thresholds trigger notification. Document in the PRD, test in evals. (See 3.7)

10. Plan your hybrid stack now. Even if running entirely on proprietary APIs today, test open-source models on lower-stakes sub-tasks. Build architecture so you can swap models per agent role. (See 3.8, 3.9)

5. Contrarian / Non-Obvious Takes

Smaller agent teams with better orchestration beat larger teams with smarter agents. Sara names the chaos problem explicitly: excellent individual agents can produce chaotic outputs without coordination. The bottleneck in agentic systems is almost never model intelligence; it is system architecture. A well-designed orchestrator with clear routing logic and quality gates consistently outperforms a collection of excellent specialists running uncoordinated. This reframes where practitioners should invest their time: not in prompt-tuning individual agents, but in designing the handoff logic between them.

LLMs should not directly output numbers in evaluation rubrics. Tyler's insight from the retail buyer intent paper is directly counterintuitive to how most people build evaluation systems. The instinct is to have an LLM rate on a 1-5 scale for simplicity. The research shows this produces skewed distributions (midpoint concentration) because of how LLMs process numeric ranges. The text-explanation-to-embedding-to-similarity approach is less intuitive but produces significantly more reliable scores. This has implications beyond evaluations: any system using LLM-generated numeric scores (lead scoring, content rating, NPS prediction) likely suffers the same midpoint bias.

The open-source timeline is shorter than most practitioners expect. Tyler projects that by May 2026, open-source models will compete with today's best proprietary models, runnable nearly for free on your own infrastructure. This collapses the token cost argument for keeping workflows simple and makes the hybrid stack the default architecture within months. The strategic implication: practitioners who build model-agnostic architectures now (see 3.9) will be positioned to capture this cost collapse; those coupled to a single vendor will pay a switching tax.

---

6. What to Revisit

Timestamps 13:00-14:50 (Tyler's co-pilot demo story): The clearest illustration of where the tool era ends and the orchestration era begins. Return to this when thinking about how to brief a co-pilot agent effectively, and more importantly, what upstream work makes that briefing credible.

Timestamps 57:50-64:00 (Tyler's deep dive on progressive evaluations): The most technically dense and practically valuable section. The LLM numeric midpoint insight and the semantic similarity fix are directly applicable to any agent evaluation workflow.

Timestamps 50:10-54:25 (Sara on knowledge architecture and graph RAG): The Cairns architecture argument is presented more quickly than it deserves. Return here when structuring memory and retrieval for production agents, particularly for temporal reasoning use cases.

Timestamps 77:00-96:00 (the Melissa Chen demo walkthrough): The best complete example of a production multi-agent system. Tyler walks through each agent's role, inter-agent communication, the orchestrator's decision logic, and outputs. Reference section for designing your own agent team architecture.

Timestamps 67:50-72:45 (model selection strategy): The Ferrari-first principle and the "27 GPT-4o mini to one Sonnet" math. Worth revisiting before any new agent project where cost is a consideration. The controlled chaos principle is a practical technique, not just a philosophy.

---

7. Sources and Further Reading

- [AI Build Lab on Maven](https://maven.com/sara-davison/ai-build-lab) -- Full course catalog including foundations and the advanced cohort described in this session. Start here to build the skills they describe.
- [Forward-Deployed Engineers Emerge as One of AI's Fastest-Growing Jobs (PYMNTS)](https://www.pymnts.com/artificial-intelligence-2/2026/forward-deployed-engineers-emerge-as-one -- Covers 800-1000% growth in FDE job listings, compensation data ($238K average total comp), and specific skills required in 2026.
- [The Forward Deployed AI Engineer: Architecting the Last Mile of AI (Sundeep Teki)](https://www.sundeepteki.org/advice/forward-deployed-ai-engineer) -- Practitioner guide to the FDE role with deeper coverage of agentic orchestration, RAG, and evaluation tooling (LangSmith, Braintrust).

- [Multi-Agent Frameworks Explained for Enterprise AI Systems (Adopt AI)](https://www.adopt.ai/blog/multi-agent-frameworks) -- 2025-2026 landscape of multi-agent orchestration frameworks (LangGraph, CrewAI); source for 72% enterprise adoption (up from 23% in 2024).
- [Building a Golden Dataset for AI Evaluation (Maxim AI)](https://www.getmaxim.ai/articles/building-a-golden-dataset-for-ai-evaluation-a-step-by-step-guide/) -- Step-by-step guide to constructing the evaluation dataset Tyler describes; pairs with the progressive evaluation framework in 3.5.
- [LangChain State of AI Agents 2025](https://www.langchain.com/state-of-agent-engineering) -- Source for 57.3% of organizations with agents in production, quality cited as top deployment barrier by 32%. Context for the evaluation-first philosophy.
- [Is RAG Dead? The Rise of Context Engineering (Towards Data Science)](https://towardsdatascience.com/beyond-rag/) -- The transition from basic RAG to context engineering and graph RAG that Sara describes. Explains why cramming all data into context is costly and impairs reasoning.
- [RAG in 2025: Enterprise Guide to Graph RAG and Agentic AI (Data Nucleus)](https://datanucleus.dev/rag-and-agentic-ai/what-is-rag-enterprise-guide-2025) -- Practical guide to agentic RAG architectures and Gartner's recommendation for semantic techniques and knowledge graphs.
- [GraphRAG and Agentic Architecture with Neo4j](https://neo4j.com/blog/developer/graphrag-and-agentic-architecture-with-neoconverse/) -- Technical deep-dive on combining graph databases with agentic architectures. Implementation reference for the multi-layer knowledge architecture in 3.6.
- [Agentic AI and Customer Success (Gainsight)](https://www.gainsight.com/blog/agentic-ai-and-customer-success-redefining-the-journey/) -- Customer success applications of agentic AI and proactive churn prevention that the Melissa Chen demo demonstrates at small-team scale.
- [Agentic AI in Real-Time Marketing (Cognizant)](https://www.cognizant.com/us/en/insights/insights-blog/agentic-ai-human-in-the-loop-approach-t -- Growth marketing applications of agentic AI with human-in-the-loop design patterns.
- [LangGraph Multi-Agent Workflows (LangChain Blog)](https://blog.langchain.com/langgraph-multi-agent-workflows/) -- Graph-based approach to multi-agent orchestration; useful for going beyond no-code to full-stack agent systems with explicit state management.

- [Kyle Poyar / Growth Unhinged: State of B2B Monetization 2025](https://growthunhinged.com/p/2025-state-of-b2b-monetization) -- Pricing upheaval data: 1,800+ pricing changes, credit models up 126% YoY, hybrid pricing as the emerging standard.